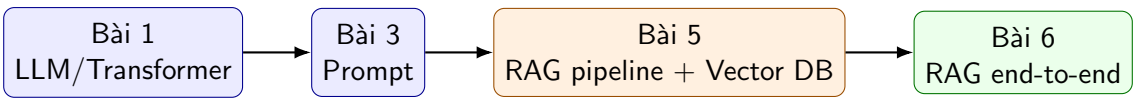
Bài 5: Kiến trúc RAG pipeline, Embedding, Semantic Search & Vector Database

AI Engineer – Module 3: AI ứng dụng

Lecture deck – 2,5 giờ

Vị trí của Bài 5 trong Module 3

- Bài 1 & 3: hiểu LLM và kiểm soát đầu ra bằng prompt.
- Bài 5: học kiến trúc RAG pipeline và cách tìm đúng tri thức trước khi yêu cầu LLM trả lời.
- Bài 6: ghép retrieval với LLM bằng LangChain/LlamaIndex để xây RAG end-to-end.

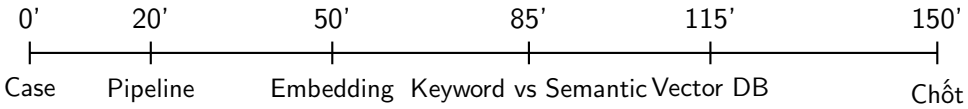


Mục tiêu đầu ra của Bài 5

Sau buổi học, học viên có thể:

1. Vẽ kiến trúc RAG pipeline: **Ingestion** → **Chunking** → **Embedding** → **Retrieval** → **Generation**.
2. Giải thích embedding như biểu diễn ý nghĩa của văn bản.
3. Phân biệt keyword search và semantic search.
4. So sánh và lựa chọn **Vector Database: FAISS, Chroma, Milvus**.
5. Nhận diện các trường hợp RAG không phải giải pháp phù hợp.

Lộ trình 150 phút phần lecture



Không học framework trước; học kiến trúc pipeline và bản chất retrieval trước.

Case mở đầu: LLM không biết tài liệu nội bộ

Tình huống: Kỹ thuật viên hỏi:

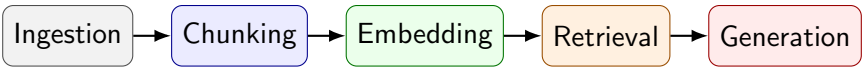
> *"Site HN-023 có alarm mất đồng bộ. Theo sổ tay vận hành phiên bản mới, cần kiểm tra bước nào trước?"*

- LLM có thể trả lời nghe hợp lý.
- Nhưng nếu model không thấy sổ tay nội bộ, câu trả lời có thể sai.
- Bài toán thật: **tìm đúng bằng chứng trước, rồi mới trả lời**.

Ba cách xử lý cùng một câu hỏi

Nhét vào prompt	Fine-tuning	Retrieval / RAG
Nhanh, dễ thử nhưng giới hạn context và khó quản lý tài liệu lớn.	Thay đổi hành vi/model style, không phải cách tốt để cập nhật tài liệu liên tục.	Tìm đoạn liên quan từ nguồn tin cậy rồi đưa vào context cho LLM.

Kiến trúc RAG pipeline tổng thể



- **Ingestion**: thu thập, làm sạch, chuẩn hóa tài liệu nguồn.
- **Chunking**: chia tài liệu thành đoạn nhỏ có ngữ cảnh.
- **Embedding**: biến mỗi chunk thành vector nghĩa.
- **Retrieval**: tìm chunk liên quan nhất với câu hỏi.
- **Generation**: LLM tổng hợp câu trả lời có căn cứ (đi sâu ở Bài 6).

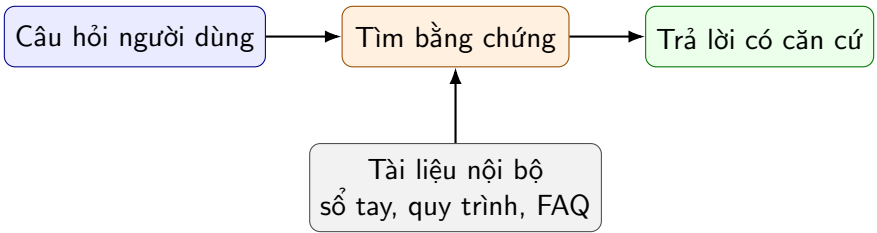
Vì sao cần retrieval?

- Tài liệu nội bộ thay đổi liên tục.
- Câu trả lời cần có nguồn chứng minh.
- Không thể đưa toàn bộ tài liệu vào prompt.
- Người dùng hỏi bằng ngôn ngữ tự nhiên, không biết chính xác từ khóa.

Ý chính

RAG bắt đầu từ retrieval tốt. Retrieval kém thì LLM rất dễ trả lời sai hoặc thiếu căn cứ.

Tư duy "nguồn sự thật"



LLM không phải nguồn sự thật; tài liệu được kiểm soát mới là nguồn sự thật.

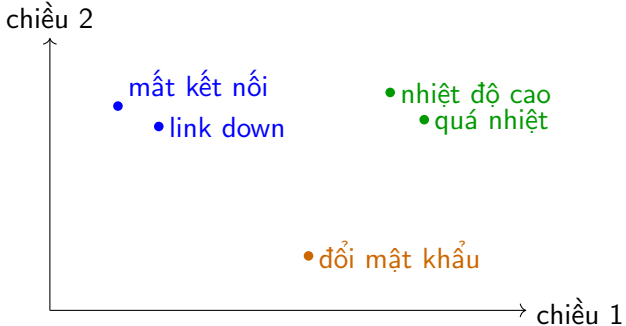
Từ search truyền thống đến semantic search

- Search truyền thống thường phụ thuộc vào từ khóa xuất hiện trong tài liệu.
- Semantic search tìm theo ý nghĩa gần nhau, kể cả khi từ ngữ khác nhau.
- Embedding là cầu nối giữa ngôn ngữ tự nhiên và phép đo độ gần ngữ nghĩa.

Embedding là gì?

Embedding là cách biến văn bản thành một biểu diễn số sao cho:

- các câu gần nghĩa nằm gần nhau,
- các câu khác nghĩa nằm xa nhau,
- ta có thể đo mức liên quan bằng khoảng cách hoặc độ tương đồng.



Embedding không phải "hiểu" như con người

- Embedding là biểu diễn thống kê học từ dữ liệu.
- Nó hữu ích cho tìm kiếm, gom cụm, phân loại gần đúng.
- Nó có thể sai khi ngữ cảnh thiếu, thuật ngữ hiếm, viết tắt mơ hồ hoặc dữ liệu khác domain.

Cách nói cho nghiệp vụ

Embedding giúp hệ thống "tìm đoạn có ý giống câu hỏi", không đảm bảo đoạn đó đủ để ra quyết định.

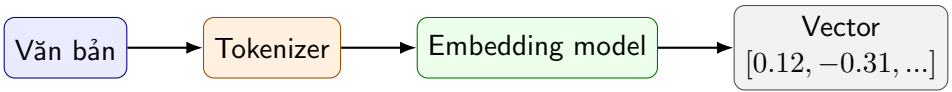
Ví dụ: gần nghĩa nhưng khác từ khóa

Câu hỏi "Thiết bị thường xuyên mất kết nối vào ban đêm thì kiểm tra gì?"

Đoạn tài liệu "Nếu link down lặp lại theo khung giờ, kiểm tra nguồn điện, lịch bảo trì và log port."

Keyword search có thể bỏ lỡ vì không có cùng từ. Semantic search có thể thấy hai đoạn này liên quan.

Quy trình tạo embedding



Trong thực hành, ta thường gọi một model/API embedding đã có sẵn (vd: từ Hugging Face Hub – Bài 3).

Embedding dùng ở đâu?

- Semantic search trên tài liệu.
- Tìm ticket tương tự.
- Gợi ý FAQ gần nhất.
- Gom cụm phản hồi khách hàng.
- Bước retrieval trong RAG.

Đo độ gần: similarity

- Sau khi có vector, ta cần xếp hạng tài liệu theo độ gần với câu hỏi.
- Metric thường gặp: cosine similarity hoặc khoảng cách vector.
- Trong lớp này chỉ cần hiểu: **điểm cao hơn nghĩa là liên quan hơn theo model embedding**.

Embedding quality phụ thuộc vào gì?

- Model embedding có phù hợp tiếng Việt/domain kỹ thuật không.
- Dữ liệu tài liệu có sạch, đúng phiên bản, ít nhiễu không.
- Chunk có giữ đủ ngữ cảnh không.
- Query của người dùng có quá mơ hồ không.

Keyword search: điểm mạnh

- Rất tốt khi cần tìm mã thiết bị, mã lỗi, tên site, số ticket.
- Dễ giải thích: có từ khóa thì match.
- Nhanh, rẻ, quen thuộc trong hệ thống hiện có.

Ví dụ

Tìm “ERR-504”, “HN-023”, “OLT-MA5600”, “port 1/1/3”.

Keyword search: điểm yếu

- Dễ bỏ lỡ khi người dùng dùng từ khác với tài liệu.
- Khó xử lý đồng nghĩa, diễn đạt dài, lỗi chính tả, viết tắt không chuẩn.
- Có thể trả nhiều kết quả chứa từ nhưng không đúng ý định.

Semantic search: điểm mạnh

- Tìm được đoạn gần nghĩa dù không trùng từ khóa.
- Phù hợp câu hỏi tự nhiên, tài liệu dài, FAQ đa dạng.
- Là nền tảng quan trọng của RAG.

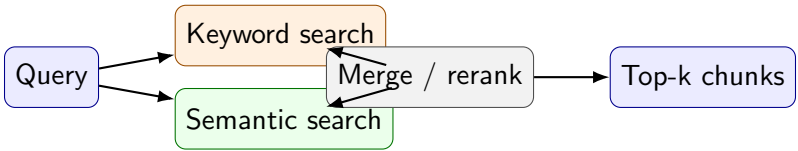
Semantic search: điểm yếu

- Có thể bỏ lỡ mã cụ thể nếu mã hiếm hoặc không được embedding tốt.
- Có thể lấy đoạn "nghe gần" nhưng thiếu điều kiện quan trọng.
- Khó giải thích hơn keyword search nếu không hiển thị chunk được lấy.

So sánh nhanh

Tiêu chí	Keyword search	Semantic search
Tìm mã/số hiệu	Rất tốt	Có thể không ổn định
Tìm theo ý nghĩa	Hạn chế	Tốt hơn
Dễ giải thích	Cao	Cần xem chunk/score
Chi phí	Thấp	Cần embedding/index
RAG	Hữu ích khi kết hợp	Thành phần cốt lõi

Hybrid search: thường là lựa chọn thực dụng



Dùng keyword cho mã/số hiệu, semantic cho ý nghĩa.

Hoạt động nhanh: chọn search strategy

Với mỗi truy vấn, chọn keyword, semantic hay hybrid:

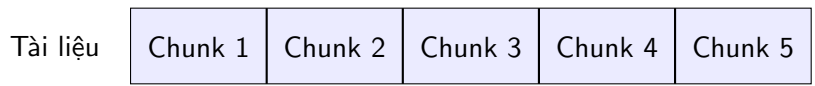
1. "ERR-504 trên site HN-023"
2. "Cách xử lý khi thiết bị chập chờn ban đêm"
3. "Sổ tay phiên bản 2024 quy định SLA thế nào?"
4. "Tìm ticket giống tình huống này"

Từ tài liệu dài đến chunk

LLM/RAG hiếm khi xử lý cả tài liệu lớn nguyên khối.

- Tài liệu được chia thành các đoạn nhỏ hơn gọi là chunk.
- Mỗi chunk được embedding và lưu vào index.
- Khi hỏi, hệ thống lấy một số chunk liên quan nhất.

Chunking trực giác



Chunk tốt giữ đủ ngữ cảnh để người đọc hiểu, nhưng không quá dài làm nhiễu retrieval. (Kỹ thuật chunking chi tiết ở Bài 6.)

Chunk quá nhỏ

- Dễ mất ngữ cảnh: câu trả lời nằm ở đoạn trước/sau.
- Retrieval có thể lấy nhiều mảnh rời rạc.
- LLM nhận context thiếu điều kiện.

Dấu hiệu

Top-k chunk có từ liên quan nhưng không đủ thông tin để trả lời.

Chunk quá lớn

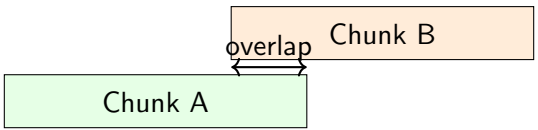
- Mỗi chunk chứa nhiều chủ đề khác nhau.
- Similarity score kém sắc nét.
- Context đưa cho LLM dài, tốn token và dễ nhiễu.

Dấu hiệu

Retriever lấy đúng tài liệu nhưng đoạn đưa vào quá rộng, LLM trích nhầm phần.

Overlap dùng để làm gì?

- Overlap giữ một phần thông tin lặp giữa hai chunk liền nhau.
- Giảm rủi ro câu/ý quan trọng bị cắt ở biên chunk.
- Overlap quá lớn làm tăng số chunk và chi phí lưu/tìm kiếm.

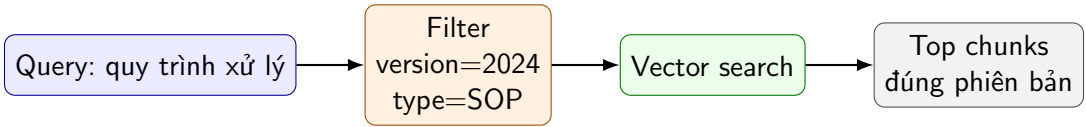


Metadata: thông tin đi kèm chunk

Metadata giúp lọc và giải thích kết quả.

- Tên tài liệu, phiên bản, ngày hiệu lực.
- Loại tài liệu: SOP, FAQ, incident report, policy.
- Site, thiết bị, miền nghiệp vụ.
- Quyền truy cập: public/internal/confidential.

Metadata filter



Không phải mọi chunk liên quan đều được phép dùng; còn phải đúng version và quyền truy cập.

Vector store là gì?

Vector store là nơi lưu:

- vector embedding của từng chunk,
- nội dung chunk gốc,
- metadata,
- cơ chế tìm nearest neighbors theo độ tương đồng (ANN).

Ví dụ công cụ: FAISS, Chroma, Milvus, Pinecone, Weaviate, Elasticsearch vector search.

Vector store không phải toàn bộ hệ thống RAG

- Nó chỉ là thành phần lưu và tìm vector.
- RAG còn cần ingestion, chunking, embedding, retrieval controls, prompt grounding, evaluation.
- Đừng học tên framework trước khi hiểu pipeline.

So sánh FAISS, Chroma, Milvus

Tiêu chí	**FAISS**	**Chroma**	**Milvus**
Bản chất	Thư viện ANN (Meta)	Vector DB nhẹ, dev-friendly	Vector DB phân tán
Triển khai	In-process, tự quản lý	Local / embedded / server	Server, scale-out
Metadata filter	Tự xử lý thêm	Có sẵn, đơn giản	Mạnh, nhiều kiểu index
Persistence	Tự lưu/nạp index	Có sẵn	Có sẵn, bền vững
Quy mô	Nhỏ–trung, rất nhanh	PoC đến vừa	Lớn, hàng tỷ vector
Phù hợp	Nhúng vào app, prototyping	PoC/RAG nhỏ, học tập	Production quy mô lớn

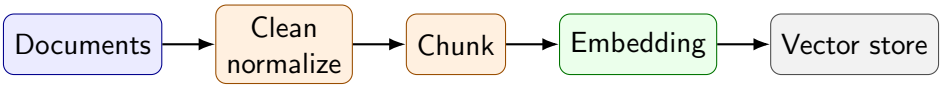
Chọn Vector Database thế nào?

- **PoC / lab / tập nhỏ**: FAISS hoặc Chroma – dễ cài, chạy local, không cần hạ tầng.
- **Cần metadata filter + persistence tiện**: Chroma cho trải nghiệm nhanh.
- **Production quy mô lớn, nhiều người dùng, dữ liệu rất lớn**: Milvus (hoặc dịch vụ quản lý tương đương).
- Tiêu chí: quy mô dữ liệu, yêu cầu latency, metadata/filter, khả năng vận hành và chính sách hạ tầng nội bộ.

Nguyên tắc

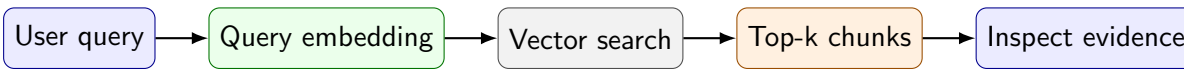
Bắt đầu đơn giản (FAISS/Chroma) để kiểm chứng use case; chỉ chuyển sang hệ phân tán khi thực sự cần quy mô.

Ingestion pipeline



Đây là phần chuẩn bị dữ liệu trước khi người dùng đặt câu hỏi.

Query pipeline



Bài 5 dừng ở query pipeline và kiểm tra chunk. Bài 6 mới ghép LLM để sinh câu trả lời.

Top-k nghĩa là gì?

- Top-k là số lượng chunk liên quan nhất được lấy ra.
- k thấp: nhanh, ít nhiễu nhưng dễ thiếu bằng chứng.
- k cao: nhiều thông tin hơn nhưng có thể gây nhiễu và tốn token.

Similarity threshold

- Có thể đặt ngưỡng điểm tương đồng tối thiểu.
- Nếu không chunk nào vượt ngưỡng, hệ thống nên nói "không tìm thấy bằng chứng đủ tốt".
- Ngưỡng quá cao: bỏ lỡ thông tin; quá thấp: lấy nhiều đoạn không liên quan.

Retrieval result cần được nhìn thấy

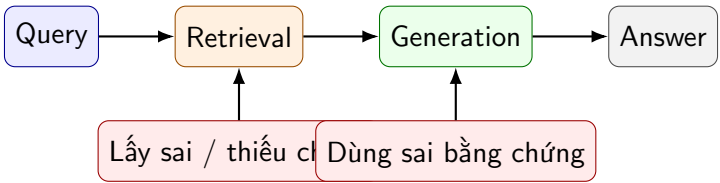
Một lỗi phổ biến: chỉ xem câu trả lời cuối cùng của LLM.

- Hãy xem top-k chunk được lấy.
- Kiểm tra chunk có trả lời được câu hỏi không.
- Tách lỗi retrieval khỏi lỗi generation.

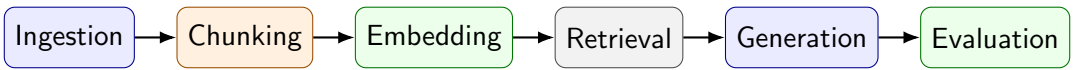
Quy tắc trong Module 3

Không đánh giá RAG nếu không xem bằng chứng được retrieval.

Tách lỗi retrieval và lỗi generation



RAG pipeline đầy đủ ở mức khái niệm



Bài 5 tập trung vào 4 bước đầu: ingestion, chunking, embedding, retrieval.

RAG không phải “LLM biết thêm”

- RAG đưa bằng chứng vào context tại thời điểm trả lời.
- Model không được huấn luyện lại chỉ vì dùng RAG.
- Nếu tài liệu thay đổi, ta cập nhật index thay vì fine-tune model.

Khi nào RAG phù hợp?

- Có kho tài liệu đáng tin cậy và thường xuyên cập nhật.
- Câu hỏi cần trích dẫn nguồn hoặc căn cứ nội bộ.
- Người dùng hỏi tự nhiên, không biết chính xác từ khóa.
- Chấp nhận thiết kế bước kiểm tra chất lượng retrieval.

Khi nào RAG không phù hợp?

- Không có tài liệu đáng tin cậy.
- Câu trả lời cần tính toán hoặc hành động, không chỉ tra cứu.
- Dữ liệu quá nhạy cảm nhưng chưa có kiểm soát quyền truy cập.
- Cần quyết định rủi ro cao mà không có human review.
- Vấn đề có thể giải bằng rule/search đơn giản hơn.

RAG so với fine-tuning

Nhu cầu	RAG	Fine-tuning
Thêm tri thức cập nhật	Phù hợp	Không phải lựa chọn chính
Thay đổi format/hành vi ổn định	Có thể hỗ trợ bằng prompt	Có thể phù hợp
Cần dẫn nguồn	Phù hợp	Không tự tạo citation
Dữ liệu thay đổi thường xuyên	Cập nhật index	Tốn quy trình huấn luyện lại

RAG so với prompt dài

- Prompt dài nhanh cho demo nhỏ.
- RAG tốt hơn khi kho tài liệu lớn, nhiều phiên bản, nhiều người dùng.
- Prompt dài không giải quyết tốt phân quyền, versioning và cập nhật dữ liệu.

Failure point 1: tài liệu đầu vào kém

- Tài liệu lỗi thời, trùng lặp, mâu thuẫn.
- PDF scan/OCR kém chất lượng.
- Thiếu metadata phiên bản hoặc quyền truy cập.

Hậu quả

Retriever có thể tìm đúng "đoạn sai" vì nguồn dữ liệu đã sai.

Failure point 2: chunking sai

- Cắt mất bảng, tiêu đề, bước quy trình.
- Cắt giữa danh sách điều kiện.
- Chunk trộn nhiều chủ đề không liên quan.

Failure point 3: embedding không phù hợp domain

- Từ viết tắt kỹ thuật không phổ biến.
- Tiếng Việt pha tiếng Anh, mã thiết bị, log message.
- Model embedding không được tối ưu cho tài liệu kỹ thuật nội bộ.

Failure point 4: query mơ hồ

- "Lỗi này xử lý sao?" nhưng không nêu lỗi nào.
- "site này" nhưng không có site id.
- "phiên bản mới" nhưng không rõ version nào.

Kiểm soát

Hệ thống nên hỏi lại hoặc yêu cầu thông tin tối thiểu trước khi retrieval.

Failure point 5: top-k không đủ hoặc quá nhiều

- Top-k quá thấp: thiếu đoạn quan trọng.
- Top-k quá cao: lẫn nhiều đoạn nhiễu.
- Cần test với bộ câu hỏi chuẩn, không chỉnh bằng cảm giác.

Đánh giá semantic search ở mức cơ bản

Với mỗi câu hỏi kiểm thử, ghi:

- Top-1 có đúng không?
- Top-3 có chứa bằng chứng đúng không?
- Chunk đúng nhưng thiếu ngữ cảnh không?
- Có chunk sai nhưng điểm cao không?

Bảng error analysis cho retrieval

Query	Kết quả lấy được	Lỗi chính	Hướng xử lý
Q1	Thiếu bước 3	Chunk quá nhỏ	Tăng chunk/overlap
Q2	Sai phiên bản	Thiếu metadata filter	Thêm version filter
Q3	Không thấy mã lỗi	Keyword cần thiết	Dùng hybrid search

Thiết kế bộ câu hỏi benchmark nhỏ

Một bộ benchmark cho Lab nên có:

- 5–10 câu hỏi đại diện cho nhu cầu thật.
- Một số câu hỏi có từ khóa chính xác.
- Một số câu hỏi dùng diễn đạt gần nghĩa.
- Một số câu hỏi cố tình thiếu thông tin hoặc ngoài phạm vi tài liệu.

Câu hỏi ngoài phạm vi cũng quan trọng

Nếu tài liệu không có câu trả lời, hệ thống tốt cần nhận diện được điều đó.

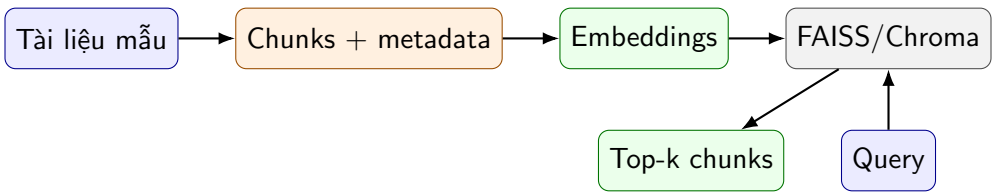
- Không retrieval được chunk đủ liên quan.
- Không nên ép LLM trả lời bằng suy đoán.
- Cần thông báo: chưa tìm thấy căn cứ trong kho tài liệu.

Quyền truy cập trong retrieval

- Không phải người dùng nào cũng được xem mọi chunk.
- Retrieval phải tôn trọng phân quyền trước khi đưa context cho LLM.
- Nếu không kiểm soát, RAG có thể làm lộ dữ liệu nội bộ (nhắc lại Bài 2).



Mini-architecture cho semantic search PoC



Checklist trước khi chuyển sang RAG

1. Tài liệu nguồn có đáng tin và đúng phiên bản không?
2. Chunk có giữ đủ ngữ cảnh không?
3. Embedding model có phù hợp ngôn ngữ/domain không?
4. Top-k retrieval đã được kiểm thử chưa?
5. Có metadata và phân quyền cơ bản chưa?

Knowledge check

Câu 1 – Câu nào đúng nhất?

1. RAG làm model ghi nhớ vĩnh viễn tài liệu nội bộ.
2. Embedding giúp biểu diễn văn bản để tìm đoạn gần nghĩa. **(đúng)**
3. Chunk càng lớn thì retrieval càng chính xác.
4. Semantic search luôn tốt hơn keyword search.

Câu 2 – Query "ERR-504 tại site HN-023"

Keyword (có mã lỗi + site ID), semantic (tìm hướng xử lý tương tự), hoặc hybrid (cả hai). Không có một strategy tốt cho mọi tình huống.

Tổng kết Bài 5

- RAG pipeline: Ingestion $\rightarrow$ Chunking $\rightarrow$ Embedding $\rightarrow$ Retrieval $\rightarrow$ Generation.
- Embedding biến văn bản thành biểu diễn số để đo gần nghĩa; semantic + keyword bổ sung nhau.
- Chunking, metadata và vector DB quyết định chất lượng retrieval.
- Chọn Vector DB theo quy mô: FAISS/Chroma cho PoC, Milvus cho production lớn.
- Phải kiểm tra top-k chunk trước khi tin câu trả lời cuối cùng.

Bridge sang Lab và Bài 6

Lab Bài 5

Xây semantic search trên sổ tay/tri thức kỹ thuật: chunk, embedding, index (FAISS/Chroma), top-k search, so sánh keyword và semantic.

Bài 6

Ghép retriever với LLM bằng LangChain/LlamaIndex để tạo mini RAG có citation và đánh giá answer quality (faithfulness, relevancy, context recall).